অধ্যায় ৯

# DOM Manipulation, Events এবং Interactive Web Applications

## শেখার লক্ষ্য

এই অধ্যায় শেষে তুমি DOM Tree পড়তে, Elements নির্বাচন ও পরিবর্তন করতে, Event Handling করতে, Event Delegation ব্যবহার করতে, Form Validation এবং `localStorage` দিয়ে Persistent Browser Application তৈরি করতে পারবে। **Practical Project: Task Manager**—Add, Edit, Delete, Complete, Search, Filter এবং Save।

## ৯.১ Browser-এ JavaScript কীভাবে কাজ করে?

HTML হলো Document-এর কাঠামো, CSS উপস্থাপনা, আর JavaScript সেই Document-এর আচরণ নিয়ন্ত্রণ করে। Browser HTML Parse করে একটি **Document Object Model (DOM)** Tree তৈরি করে। প্রতিটি Element, Attribute এবং Text DOM-এর Node হিসেবে উপস্থাপিত হয়। DOM হলো Browser-provided API; Node.js-এ সাধারণভাবে `document` পাওয়া যায় না।

```HTML
<main id="app">
  <h1>আমার প্রথম DOM</h1>
  <p class="message">শুরু করছি...</p>
</main>
```

এই HTML-এ `document`-এর নিচে `html`, `body`, `main`, `h1`, `p`—Parent/Child সম্পর্ক রয়েছে। `main` হলো `h1` ও `p`-এর Parent। `querySelector()` দিয়ে CSS Selector ব্যবহার করে Element খুঁজে পাওয়া যায়।

```JAVASCRIPT
const heading = document.querySelector('#app h1');
const message = document.querySelector('.message');
heading.textContent = 'JavaScript Mastery';
message.textContent = 'DOM সফলভাবে পরিবর্তিত হয়েছে';
```

`querySelector()` প্রথম Matching Element অথবা `null` দেয়; `querySelectorAll()` একটি **static NodeList** দেয়। `getElementById()` ID দিয়ে Element খুঁজে পায়। `getElementsByClassName()` একটি **live HTMLCollection** দেয়। Element না পাওয়া গেলে `null` চেক করো।

```JAVASCRIPT
const button = document.querySelector('#saveButton');
if (button) button.disabled = false;
```

## ৯.২ `textContent`, `innerHTML` ও নিরাপদ Rendering

`textContent` Text হিসেবে Value বসায়; `innerHTML` String-কে HTML হিসেবে Parse করে। User Input সরাসরি `innerHTML`-এ বসানো বিপজ্জনক: HTML Injection ও Cross-Site Scripting (XSS) হতে পারে।

```javascript
const userInput = '<img src=x onerror=alert(1)>';
const output = document.querySelector('#output');
output.textContent = userInput; // text হিসেবেই দেখাবে
```

`innerHTML`-এ User Data ব্যবহার না করে `createElement()`, `textContent`, `append()` বা `replaceChildren()` ব্যবহার করো। Sanitization প্রয়োজন হলে Trusted, Maintained Library ও Context-specific Security Controls ব্যবহার করতে হবে।

## ৯.৩ Element তৈরি, যোগ ও সরানো

```javascript
const list = document.querySelector('#items');
const li = document.createElement('li');
li.textContent = 'DOM Practice';
li.classList.add('task');
list.append(li);
// পরে প্রয়োজন হলে: li.remove();
```

`append()` Text বা Node যোগ করে; `appendChild()` শুধু Node নেয়। `prepend()` শুরুতে যোগ করে। `replaceChildren()` পুরোনো Child Nodes সরিয়ে নতুনগুলো বসায়। `classList.add/remove/toggle` CSS Class নিয়ন্ত্রণ করে; `element.dataset.id` `data-id` Attribute-এর সঙ্গে যুক্ত।

## ৯.৪ Events: User Interaction-এর প্রতিক্রিয়া

Browser Click, Input, Change, Submit, Keydown-এর মতো Events তৈরি করে। `addEventListener()` একটি Function-কে Event Listener হিসেবে নিবন্ধন করে।

```javascript
const button = document.querySelector('#hello');
button.addEventListener('click', (event) => {
  console.log('Clicked:', event.currentTarget);
});
```

`event.target` হলো Event-এর উৎপত্তিস্থল; `event.currentTarget` হলো যে Element-এর Listener এখন চলছে। `event.preventDefault()` Cancelable Event-এর Default Action বন্ধ করে (যেমন Form Submit-এর Navigation)। `event.stopPropagation()` Event-এর পরবর্তী Propagation বন্ধ করে; সাধারণ ক্ষেত্রে অযথা ব্যবহার করা উচিত নয়।

### Event Propagation

Event সাধারণত **Capture → Target → Bubble** পর্যায়ে চলে। Parent Container-এ Listener রেখে Child Button-এর Event Handle করাকে **Event Delegation** বলা হয়। Dynamic List-এ প্রতিটি Button-এ Listener বসানোর বদলে Parent List-এ একটি Listener রাখা সুবিধাজনক।

```javascript
const list = document.querySelector('#taskList');
list.addEventListener('click', (event) => {
  const button = event.target.closest('button[data-action]');
  if (!button || !list.contains(button)) return;
  const row = button.closest('li[data-id]');
  if (!row) return;
  console.log(button.dataset.action, row.dataset.id);
});
```

## ৯.৫ Form Handling ও Validation

Form-এর `submit` Event Enter Key দিয়েও Trigger হয়। শুধু Button-এর `click` Listener দিলে Keyboard Flow বাদ পড়তে পারে। HTML Validation (`required`, `maxlength`) ও JavaScript Validation একসঙ্গে ব্যবহার করো। Frontend Validation User Experience বাড়ায়, কিন্তু Backend থাকলে Server-side Validation অবশ্যই প্রয়োজন।

```javascript
form.addEventListener('submit', (event) => {
  event.preventDefault();
  const title = titleInput.value.trim();
  if (!title) return;
  console.log('New task:', title);
  form.reset();
});
```

## ৯.৬ State ও UI আলাদা রাখা

State হলো App-এর Data; UI হলো সেই Data-এর Visual Representation। ভালো Design হলো `state → render() → DOM`। Event হলে Data Update হবে, তারপর `render()` DOM Update করবে। এতে Logic Testing সহজ হয়।

```javascript
let tasks = [];
function render() {
  list.replaceChildren();
  for (const task of tasks) {
    const li = document.createElement('li');
    li.textContent = task.title;
    list.append(li);
  }
}
```

বড় Application-এ Full Re-render-এর পরিবর্তে Selective Update বা Framework Rendering দরকার হতে পারে; ছোট Task Manager-এর জন্য Full Re-render যথেষ্ট।

## ৯.৭ Browser Storage: `localStorage`

`localStorage` Key/Value Pair সংরক্ষণ করে এবং Value String হয়। `JSON.stringify()` দিয়ে Array/Object String-এ রূপান্তর এবং `JSON.parse()` দিয়ে পুনরুদ্ধার করা যায়। Storage Browser/Origin অনুযায়ী আলাদা। Private Browsing, Quota, Browser Policy বা Disabled Storage-এর কারণে Access ব্যর্থ হতে পারে—তাই `try/catch` ব্যবহার করো। Password, Token বা Sensitive Personal Data `localStorage`-এ রাখা উচিত নয়।

```javascript
const KEY = 'demo-tasks';
try {
  localStorage.setItem(KEY, JSON.stringify([{ id: '1', title: 'Read DOM' }]));
  const tasks = JSON.parse(localStorage.getItem(KEY) ?? '[]');
  console.log(tasks);
} catch (error) {
  console.warn('Storage unavailable or invalid:', error);
}
```

Production Application-এ Parsed Data-এর Shape Validate করা জরুরি। `JSON.parse()` সফল হলেও Data প্রত্যাশিত Array বা Task Object নাও হতে পারে।

## ৯.৮ Practical Project — Task Manager

### Requirements

1. Task Add: ফাঁকা Title Reject, Maximum 120 Characters।
2. Task Complete/Undo: Checkbox দিয়ে Status পরিবর্তন।
3. Task Edit/Delete: প্রতিটি Row থেকে Action।
4. Search ও Filter: All / Active / Completed।
5. Summary: Total, Active, Completed Count।
6. Persistence: Reload-এর পর Saved Tasks Restore।
7. Security: Task Title DOM-এ `textContent` দিয়ে Render।
8. Accessibility: Form Label, Keyboard Submit, `aria-live` Status।

### Project Structure

```
source-code/chapter-09/
├── index.html
├── style.css
├── task-core.js
├── app.js
└── task-core.test.js
```

**Source Code:** বইয়ের Companion Source Code-এর `source-code/chapter-09/` Folder-এ সব File সম্পূর্ণ রয়েছে।

### Architecture

```
User Event → DOM Listener → TaskCore (Pure Function)
                                  ↓
                          Updated Task Array
                                  ↓
                    localStorage + render()
                                  ↓
                           Updated UI
```

`task-core.js`-এ Pure Functions রাখা হয়েছে; DOM ও Storage Code `app.js`-এ। এতে Node.js-এর Built-in Test Runner দিয়ে Core Logic আলাদাভাবে পরীক্ষা করা যায়।

### Run

1. `source-code/chapter-09/index.html` Browser-এ খুলো।
2. Task Add, Complete, Edit, Delete, Search ও Filter পরীক্ষা করো।
3. Page Reload করে Persistence পরীক্ষা করো।
4. Node.js 18+ থাকলে Terminal-এ `node --test task-core.test.js` চালাও।

**Limitation:** Browser Storage Policy ও Local File Context অনুযায়ী Behaviour ভিন্ন হতে পারে। নির্ভরযোগ্য Local Development-এর জন্য Static HTTP Server ব্যবহার করো, যেমন `python -m http.server 8000`, তারপর `http://localhost:8000`।

## ৯.৯ Debugging Checklist

| সমস্যা | সম্ভাব্য কারণ | সমাধান |
|---|---|---|
| `Cannot read properties of null` | Element Selector ভুল বা Script আগে চলছে | Script Body-এর শেষে রাখো, Selector যাচাই করো |
| Button Click কাজ করছে না | Listener/Selector ভুল | DevTools Console, Event Listener Inspect |
| Form Reload হয়ে যাচ্ছে | `preventDefault()` নেই | Submit Handler-এ যোগ করো |
| Tasks Reload-এ হারায় | Storage ব্যর্থ বা Key বদলেছে | DevTools Application Tab দেখো |
| HTML Injection | `innerHTML`-এ User Data | `textContent` ব্যবহার করো |
| Dynamic Buttons কাজ করছে না | নতুন Elements-এ Listener নেই | Event Delegation ব্যবহার করো |

## ৯.১০ Exercises

**Exercise 1:** `#title` Element-এর Text বদলে `DOM Mastery` করো।

**Exercise 2:** `ul#list`-এ নতুন `li` তৈরি করে যোগ করো।

**Exercise 3:** একটি Button Click করলে `.dark` Class Toggle করো।

**Exercise 4:** Form Submit করলে Input-এর Trimmed Value Log করো; Empty Input Reject করো।

**Exercise 5:** একটি List-এ Event Delegation দিয়ে Clicked Item-এর `data-id` পড়ো।

**Exercise 6:** Array of Tasks থেকে Completed Task Filter করো।

**Exercise 7:** `localStorage`-এ Array Save ও Restore করো; Invalid JSON Handle করো।

**Exercise 8:** `innerHTML`-এর বদলে `textContent` ব্যবহার করে User Text নিরাপদে Render করো।

**Exercise 9:** Task Manager-এ `clearCompleted()` যুক্ত করো।

**Exercise 10:** Search Query Case-insensitive করো।

## Exercise Solutions

JAVASCRIPT

```
// 1
const titleNode = document.querySelector('#title');
if (titleNode) titleNode.textContent = 'DOM Mastery';

// 2
const ul = document.querySelector('#list');
if (ul) { const li = document.createElement('li'); li.textContent = 'New item'; ul.append(li); }

// 3
const toggle = document.querySelector('#toggle');
toggle?.addEventListener('click', () => document.body.classList.toggle('dark'));

// 4
form.addEventListener('submit', e => { e.preventDefault(); const v = input.value.trim(); if (v)
console.log(v); });

// 5
list.addEventListener('click', e => { const item = e.target.closest('li[data-id]'); if (item &&
list.contains(item)) console.log(item.dataset.id); });

// 6
const completed = tasks.filter(task => task.completed);

// 7
try { localStorage.setItem('tasks', JSON.stringify(tasks)); const restored =
JSON.parse(localStorage.getItem('tasks') ?? '[]'); console.log(Array.isArray(restored) ?
restored : []); } catch { console.log([]); }

// 8
output.textContent = userProvidedText;

// 9
const activeTasks = tasks.filter(task => !task.completed);

// 10
const results = tasks.filter(task =>
task.title.toLocaleLowerCase().includes(query.trim().toLocaleLowerCase()));
```

*নোট:* Solution Snippets-এ `form`, `input`, `list`, `tasks`, `output` ইত্যাদি সংশ্লিষ্ট Exercise-এর পূর্বনির্ধারিত Variables। এগুলো একত্রে Copy করে চালানোর জন্য নয়।

## ৯.১১ MCQ Quiz

1. DOM-এর পূর্ণরূপ কী? (A) Data Object Memory (B) Document Object Model (C) Dynamic Order Method (D) Document Output Map
2. `querySelector()` Match না পেলে কী দেয়? (A) `[]` (B) `undefined` (C) `null` (D) Error
3. User Text Render-এর জন্য নিরাপদ সাধারণ পদ্ধতি? (A) `innerHTML` (B) `eval()` (C) `textContent` (D) `document.write()`
4. Form Submit-এর Default Navigation থামায় কোনটি? (A) `stop()` (B) `preventDefault()` (C) `pause()` (D) `cancel()`
5. Event Delegation-এর Listener সাধারণত কোথায় থাকে? (A) Parent (B) প্রতিটি Child-এ বাধ্যতামূলক (C) CSS (D) Server
6. `localStorage` Value-এর Type কী? (A) Array (B) Object (C) String (D) Function
7. `JSON.stringify()` কী করে? (A) JSON Text তৈরি (B) DOM তৈরি (C) CSS তৈরি (D) Network Request
8. `classList.toggle()` কী করে? (A) Remove All (B) Class Add/Remove Toggle (C) Change ID (D) Reload
9. `event.target` কী নির্দেশ করে? (A) Listener Registered Element সবসময় (B) Event-এর Originating Target (C) Browser Window সবসময় (D) CSS Class
10. Browser DOM API ছাড়া Core Logic Test সহজ করতে কী করব? (A) সব Logic Event Handler-এ (B) Pure Functions আলাদা (C) HTML-এ Inline JS (D) Tests বাদ

### Answer Key ও Explanation

1. **B** — DOM হলো Document Object Model।
2. **C** — প্রথম Matching Element না থাকলে `null`।
3. **C** — `textContent` String-কে HTML Parse করে না।
4. **B** — `preventDefault()` Cancelable Default Action থামায়।
5. **A** — Parent Container Listener Child Event Handle করতে পারে।
6. **C** — Web Storage API String Values সংরক্ষণ করে।
7. **A** — JavaScript Value থেকে JSON String তৈরি হয়।
8. **B** — নির্দিষ্ট CSS Class থাকলে সরায়, না থাকলে যোগ করে।
9. **B** — Event-এর Target এবং Listener-এর Current Target আলাদা হতে পারে।
10. **B** — Pure Logic DOM ছাড়াই Unit Test করা যায়।

## ৯.১২ অধ্যায়ের সারাংশ ও পরবর্তী অধ্যায়

DOM Selection, Secure Rendering, Events, Delegation, Form Handling, State, Storage ও Testing একত্রে একটি কার্যকর Browser App তৈরির ভিত্তি। পরের অধ্যায়ে শিখব **Asynchronous JavaScript, Promises, async/await ও Event Loop**—যা API-driven Web Application তৈরির জন্য অপরিহার্য।